Course: CS420 – Artificial Intelligence

Midterm examination review – Lectures 0 to 3

PRACTICE MIDTERM EXAMINATION – TEST 4
SOLUTION

Duration: 120 minutes · Closed book · Calculator allowed

Unless a question says otherwise, ties are broken in alphabetical order (left-to-right in trees), and successors are generated in alphabetical order.

Question 1. *Vacuum world with three cells.* A vacuum-cleaner robot lives in a row of **three** cells (1, 2, 3). Each cell is either dirty or clean. The robot can move Left, move Right, or Suck (clean its current cell).

(a) Consider a state as a tuple of values. Which values should be included in the tuple?

(ℓ, d_1, d_2, d_3): robot location $\ell \in \{1, 2, 3\}$ and the dirt status of each cell.

(b) Define the initial state (robot in cell 1, all cells dirty) and the goal state(s).

Initial $(1, D, D, D)$. Goal: any state with all cells clean, (ℓ, C, C, C) for $\ell = 1, 2, 3$ (three goal states).

(c) Define the action function. What happens when moving Left in cell 1, Right in cell 3, or Sucking a clean cell?

Left: $\ell \to \max(1, \ell - 1)$; Right: $\ell \to \min(3, \ell + 1)$; Suck: $d_\ell \to C$. Those three cases have *no effect* (same state).

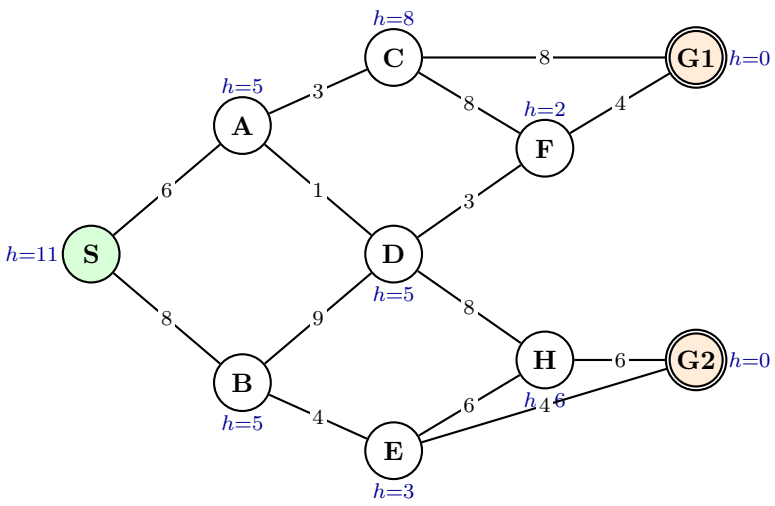
(d) What is the action cost?

1 per action.

(e) What is the maximum number of states? How many are reachable from the initial state? Give a shortest solution.

$3 \cdot 2^3 = \mathbf{24}$ states (in general $n \cdot 2^n$); all **24** are reachable from $(1, D, D, D)$. Shortest solution (5 actions): Suck, Right, Suck, Right, Suck.

Question 2. Consider the undirected graph below with initial state S and **two** goal states G_1, G_2; a search stops at whichever goal satisfies its goal test first.



For each search strategy, (a) list, in order, the states **expanded**, (b) give the path found and (c) give the final content of the frontier (a state is expanded when it is removed from the frontier). Conventions: BFS, DFS and greedy best-first search test a state for the goal when it is *generated* (so the goal is not expanded); UCS and A* test it when it is *removed* from the frontier (graph search; a cheaper path to a frontier state replaces its entry). DFS avoids loops by remembering the states on the current path only. The path returned will not be accepted if the list of expanded states is wrong.

Algorithm	**List of expanded states**	**Path returned**	**Final frontier**
Breadth-first search	S A B C	S A C G1 (cost 17)	{ D, E, F }
Uniform-cost search	S A D B C F E G1	S A D F G1 (cost 14)	{ H, G2 }
Depth-first search	S A C	S A C G1 (cost 17)	(not asked)
Greedy best-first search	S A B E	S B E G2 (cost 16)	{ D, C }
Graph-search A*	S A D F B G1	S A D F G1 (cost 14)	{ E, C, H }

BFS (queue after each expansion):

Step	Expanded	Queue (front → back) after the expansion
1	S	A, B
2	A	B, C, D
3	B	C, D, E
4	C	generates goal **G1** ⇒ stop; frontier: D, E, F

DFS (stack after each expansion):

Step	Expanded	Stack (top → bottom) after the expansion
1	S	A, B
2	A	C, D, B
3	C	generates goal **G1** ⇒ stop; frontier: D, B

UCS:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $g(n)$)
1	S_0	A_6, B_8
2	A_6	D_7, B_8, C_9
3	D_7	B_8, C_9, F_{10}, H_{15}
4	B_8	C_9, F_{10}, E_{12}, H_{15}
5	C_9	F_{10}, E_{12}, H_{15}, $G1_{17}$
6	F_{10}	E_{12}, $G1_{14}$, H_{15}
7	E_{12}	$G1_{14}$, H_{15}, $G2_{16}$
8	$G1_{14}$	*goal removed from the frontier – stop*

Greedy best-first:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $h(n)$)
1	S_{11}	A_5, B_5
2	A_5	B_5, D_5, C_8
3	B_5	E_3, D_5, C_8
4	E_3	generates goal **G2** ⇒ stop; frontier: D_5, C_8

A*:

Step	Expanded	Frontier after the expansion, in priority order (subscript $= g + h = f$)
1	$S_{0+11=11}$	$A_{6+5=11}$, $B_{8+5=13}$
2	$A_{6+5=11}$	$D_{7+5=12}$, $B_{8+5=13}$, $C_{9+8=17}$
3	$D_{7+5=12}$	$F_{10+2=12}$, $B_{8+5=13}$, $C_{9+8=17}$, $H_{15+6=21}$
4	$F_{10+2=12}$	$B_{8+5=13}$, $G1_{14+0=14}$, $C_{9+8=17}$, $H_{15+6=21}$
5	$B_{8+5=13}$	$G1_{14+0=14}$, $E_{12+3=15}$, $C_{9+8=17}$, $H_{15+6=21}$
6	$G1_{14+0=14}$	*goal removed from the frontier – stop*

(d) With several goals, what must an admissible heuristic estimate? Is h admissible and consistent?

The cost to the *nearest* goal. h^*: S= 14, A= 8, B= 8, C= 8, D= 7, E= 4, F= 4, H= 6, G1= 0, G2= 0; $h \le h^*$ (admissible) and $h(u) \le c(u, v) + h(v)$ on every edge (consistent).

(e) Which goal does each strategy reach? Why does greedy best-first search end at a different goal from A*?

BFS→G1, UCS→G1, DFS→G1, GBFS→G2, A*→G1. GBFS only follows h: after the tie A/B ($h = 5$) and then B/D (alphabetical), it expands B, whose child E leads straight to G_2 (cost 16). A* also counts the cost already paid and finds the cheaper route to G_1 (cost 14).

Question 3. Consider the 8-puzzle. Apply *steepest-descent hill climbing* with the Manhattan-distance heuristic h to the following pair of initial and goal states.

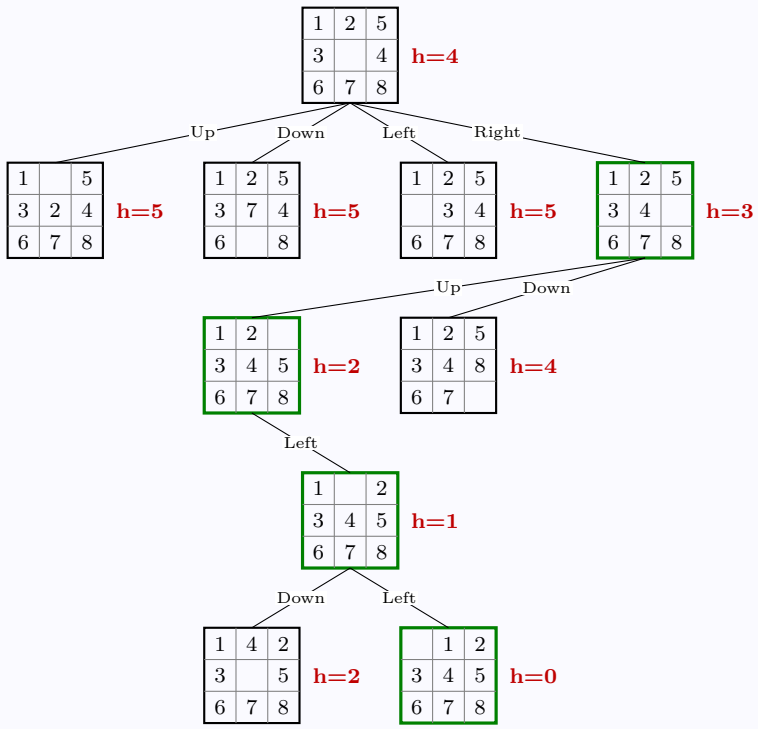
Initial state

1	2	5
3	–	4
6	7	8

Goal state

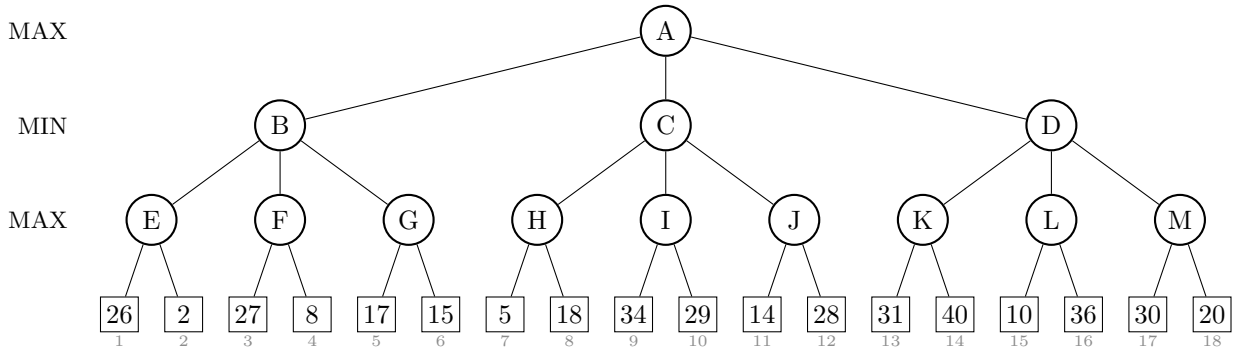
–	1	2
3	4	5
6	7	8

Your work should: draw the search tree including all successors of every expanded state (moves of the blank in the order Up, Down, Left, Right; do not regenerate the parent state), write the heuristic value of every node, and mark the path found.



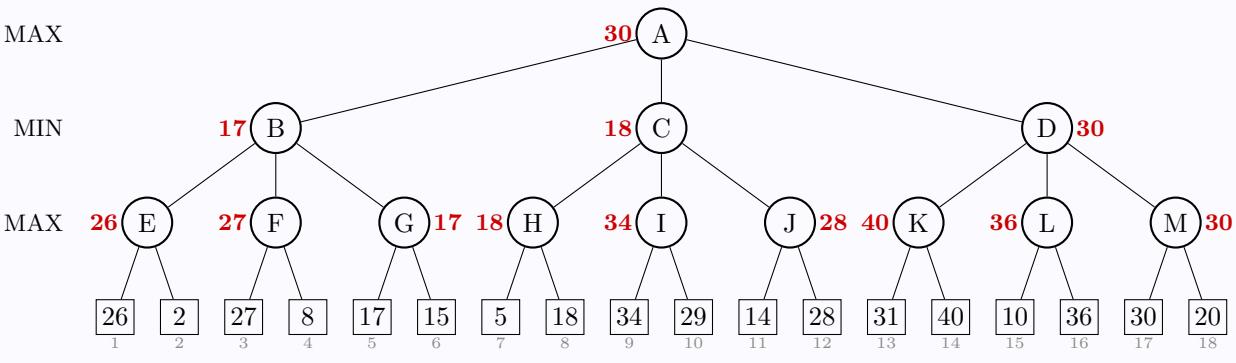
Each level keeps the successor with the smallest h (green). h decreases 4→3→2→1→0 and the goal is reached after 4 moves: Right, Up, Left, Left.

Question 4. Consider the following game tree (MAX at the root).



(a) Compute the final backed-up value of every node using the minimax algorithm (no alpha-beta pruning at this step). Which move should MAX make at the root?

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**
Minimax	30	17	18	30	26	27	17	18	34	28	40	36	30

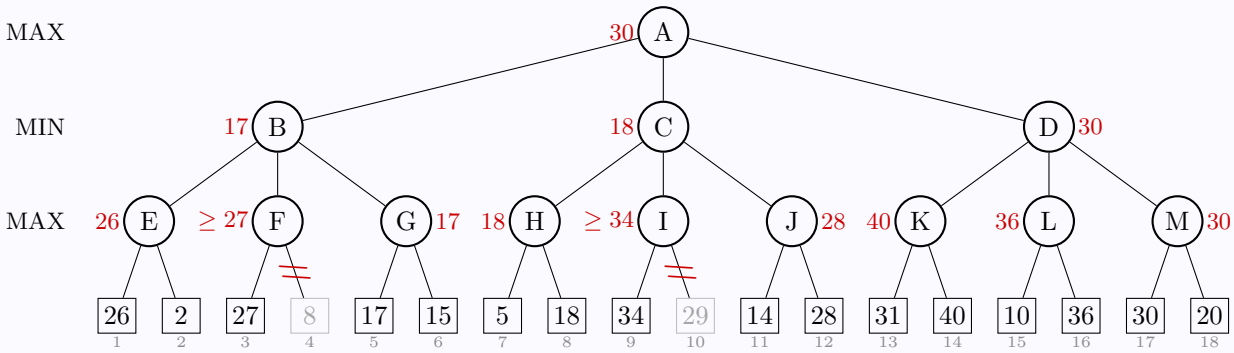


Root value = 30; MAX moves to **D**.

(b) Run alpha-beta pruning (children visited left to right). Give the value computed at every node (mark **X** if the node is never visited), cross out the pruned branches on the tree and briefly explain each case.

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**
α-β	30	17	18	30	26	≥ 27	17	18	≥ 34	28	40	36	30

($\leq v$ / $\geq v$: the node was cut off, so only this bound is known; writing the returned number v is also accepted.)



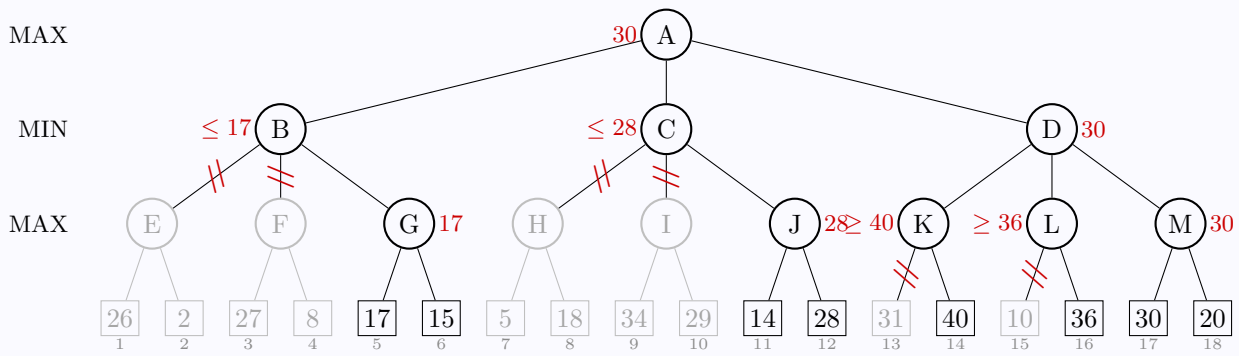
α and β at every visited node:

Node	Type	On entry	When it returns	Value returned
A	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 30,\ \beta = +\infty$	30
B	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 17$	17
E	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 26,\ \beta = +\infty$	26
F	MAX	$\alpha = -\infty,\ \beta = 26$	$\alpha = 27,\ \beta = 26$	≥ 27
G	MAX	$\alpha = -\infty,\ \beta = 26$	$\alpha = 17,\ \beta = 26$	17
C	MIN	$\alpha = 17,\ \beta = +\infty$	$\alpha = 17,\ \beta = 18$	18
H	MAX	$\alpha = 17,\ \beta = +\infty$	$\alpha = 18,\ \beta = +\infty$	18
I	MAX	$\alpha = 17,\ \beta = 18$	$\alpha = 34,\ \beta = 18$	≥ 34
J	MAX	$\alpha = 17,\ \beta = 18$	$\alpha = 28,\ \beta = 18$	28
D	MIN	$\alpha = 18,\ \beta = +\infty$	$\alpha = 18,\ \beta = 30$	30
K	MAX	$\alpha = 18,\ \beta = +\infty$	$\alpha = 40,\ \beta = +\infty$	40
L	MAX	$\alpha = 18,\ \beta = 40$	$\alpha = 36,\ \beta = 40$	36
M	MAX	$\alpha = 18,\ \beta = 36$	$\alpha = 30,\ \beta = 36$	30

- At MAX node **F**, after leaf #3: $v = 27 \geq \beta = 26 \Rightarrow$ prune leaf #4. (MIN at B already has an alternative worth $\beta = 26$, so it will never let play reach F.)
- At MAX node **I**, after leaf #9: $v = 34 \geq \beta = 18 \Rightarrow$ prune leaf #10. (MIN at C already has an alternative worth $\beta = 18$, so it will never let play reach I.)

Leaves evaluated: 16 of 18.

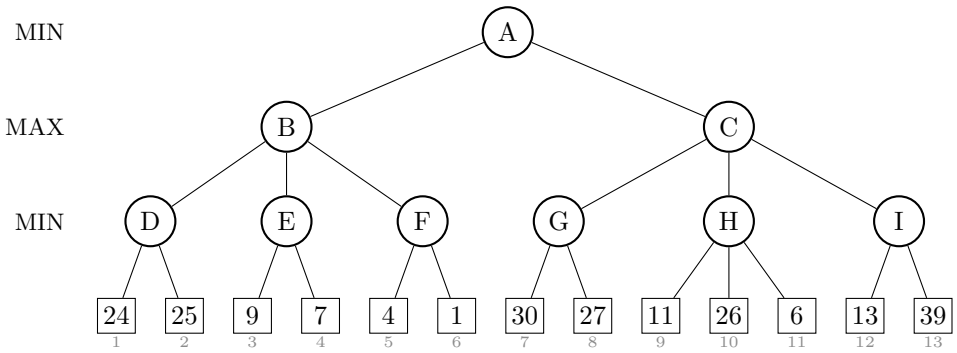
(c) Run alpha-beta again, this time visiting the children of every node from **right to left**. Which leaves are pruned now? Explain the difference.



- At MAX node **L**, after leaf #16: $v = 36 \geq \beta = 30 \Rightarrow$ prune leaf #15. (MIN at D already has an alternative worth $\beta = 30$, so it will never let play reach L.)
- At MAX node **K**, after leaf #14: $v = 40 \geq \beta = 30 \Rightarrow$ prune leaf #13. (MIN at D already has an alternative worth $\beta = 30$, so it will never let play reach K.)
- At MIN node **C**, after child J: $v = 28 \leq \alpha = 30 \Rightarrow$ prune I (leaves #9–#10), H (leaves #7–#8). (MAX at A already has an alternative worth $\alpha = 30$, so it will never let play reach C.)
- At MIN node **B**, after child G: $v = 17 \leq \alpha = 30 \Rightarrow$ prune F (leaves #3–#4), E (leaves #1–#2). (MAX at A already has an alternative worth $\alpha = 30$, so it will never let play reach B.)

Pruned leaves: #1, #2, #3, #4, #7, #8, #9, #10, #13, #15 (10) instead of 2. Going right to left, MAX first explores D, the best child (value 30), so α is high at once and the other MIN nodes are cut as soon as a child is $\leq \alpha$: good move ordering is what makes alpha-beta efficient.

Question 5. Consider the game tree below, in which **MIN moves first**.

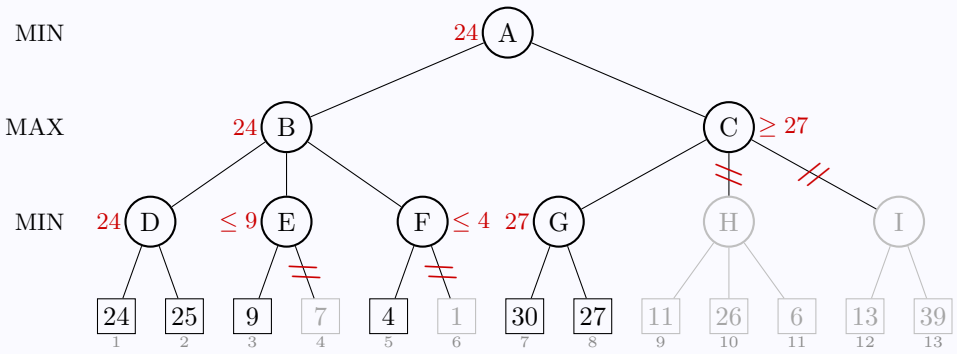


(a) Note the minimax value of every node (no pruning).

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
Minimax	24	24	27	24	7	1	27	6	13

Root value = 24; MIN moves to **B**.

(b) Show where alpha-beta pruning occurs (left to right) by crossing out the corresponding branches, and briefly explain each case.

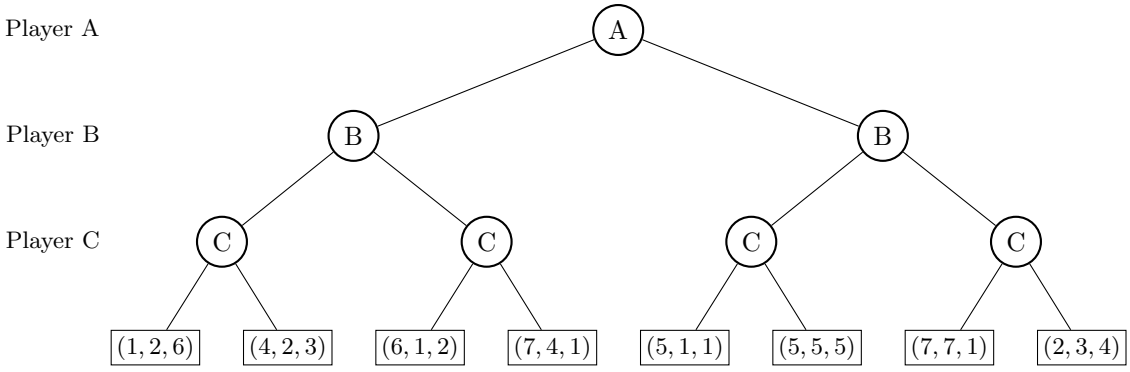


Node	Type	On entry	When it returns	Value returned
A	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 24$	24
B	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 24,\ \beta = +\infty$	24
D	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 24$	24
E	MIN	$\alpha = 24,\ \beta = +\infty$	$\alpha = 24,\ \beta = 9$	≤ 9
F	MIN	$\alpha = 24,\ \beta = +\infty$	$\alpha = 24,\ \beta = 4$	≤ 4
C	MAX	$\alpha = -\infty,\ \beta = 24$	$\alpha = 27,\ \beta = 24$	≥ 27
G	MIN	$\alpha = -\infty,\ \beta = 24$	$\alpha = -\infty,\ \beta = 24$	27

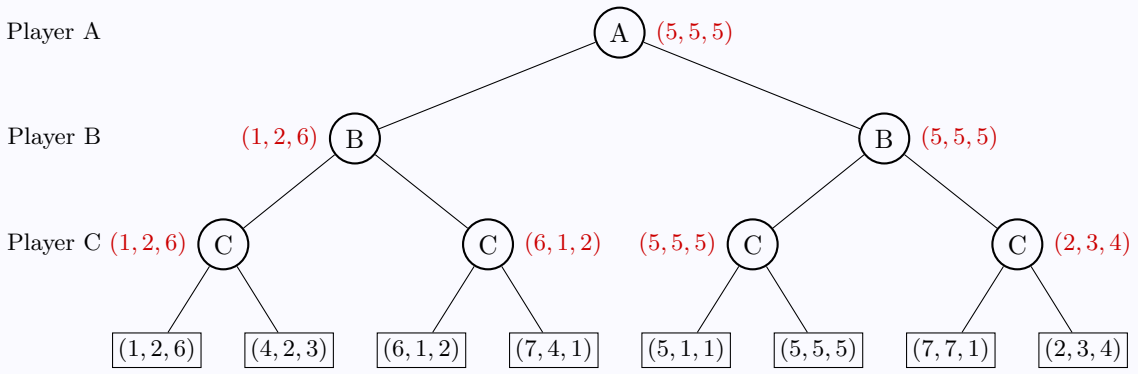
- At MIN node **E**, after leaf #3: $v = 9 \leq \alpha = 24 \Rightarrow$ prune leaf #4. (MAX at B already has an alternative worth $\alpha = 24$, so it will never let play reach E.)
- At MIN node **F**, after leaf #5: $v = 4 \leq \alpha = 24 \Rightarrow$ prune leaf #6. (MAX at B already has an alternative worth $\alpha = 24$, so it will never let play reach F.)
- At MAX node **C**, after child G: $v = 27 \geq \beta = 24 \Rightarrow$ prune H (leaves #9–#11), I (leaves #12–#13). (MIN at A already has an alternative worth $\beta = 24$, so it will never let play reach C.)

Leaves evaluated: 6 of 13; never evaluated: #4, #6, #9, #10, #11, #12, #13.

Question 6. Three players A, B, C move in turn. Each leaf gives the utility vector (u_A, u_B, u_C) and every player maximises its own component.



(a) Back up the utility vectors to every node. Which move does A make?

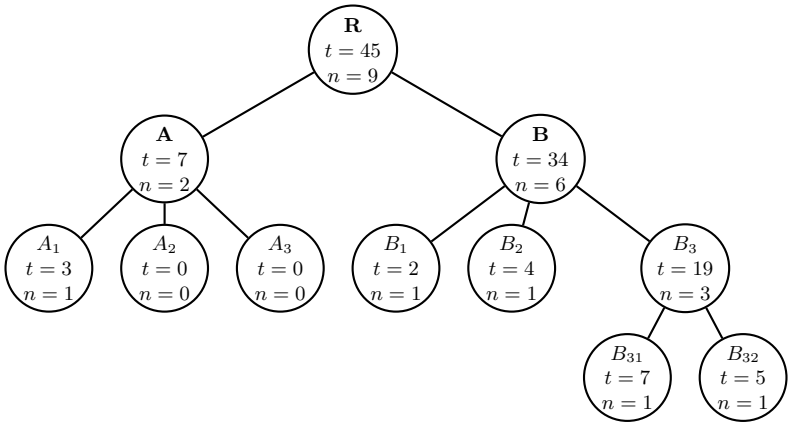


Root $= (5, 5, 5)$: A moves to the right subtree (A gets 5 instead of 1).

(b) The game is not zero-sum. Explain, using this tree, why alliances may form and why they may be broken.

$(5, 5, 5)$ is good for everybody, so A and B both prefer the right subtree: collaboration can arise. But each player keeps an alliance only while it serves its own utility: at the right-most C node, C prefers $(2, 3, 4)$ to $(7, 7, 1)$, which A and B would both like.

Question 7. Consider the MCTS tree below; rewards are scores in $[0, 10]$ for the root player. Use UCB1 $= \bar{V}_i + C\sqrt{\ln N / n_i}$ with $C = 2$ (unvisited $= \infty$, ties to the left; a never-visited leaf is rolled out directly, otherwise all its children are added and the rollout starts at the first). Expansion: $A \to A_1, A_2, A_3$; $B \to B_1, B_2, B_3$; $X_i \to X_{i1}, X_{i2}$; $X_{ij} \to X_{ij1}, X_{ij2}$.



(a) Perform three iterations whose rollouts return 0, 1, 10 (in order). Give the UCB1 values at every selection step and draw the final tree.

Iteration 1.

- *Selection* at R ($N = 9$): A: $\frac{7}{2} + 2\sqrt{\frac{\ln 9}{2}} = 5.596$; B: $\frac{34}{6} + 2\sqrt{\frac{\ln 9}{6}} = 6.877 \Rightarrow$ go to **B**.
- *Selection* at B ($N = 6$): B_1: $\frac{2}{1} + 2\sqrt{\frac{\ln 6}{1}} = 4.677$; B_2: $\frac{4}{1} + 2\sqrt{\frac{\ln 6}{1}} = 6.677$; B_3: $\frac{19}{3} + 2\sqrt{\frac{\ln 6}{3}} = 7.879$ $\Rightarrow$ go to B_3.
- *Selection* at B_3 ($N = 3$): B_{31}: $\frac{7}{1} + 2\sqrt{\frac{\ln 3}{1}} = 9.096$; B_{32}: $\frac{5}{1} + 2\sqrt{\frac{\ln 3}{1}} = 7.096 \Rightarrow$ go to B_{31}.
- *Expansion*: B_{31} was visited before ($n > 0$) and has no children $\Rightarrow$ add B_{311}, B_{312}; roll out from the first one, B_{311}.
- *Simulation*: rollout from B_{311} returns 0.
- *Backpropagation*: add 0 to t and 1 to n for R $\to$ B $\to B_3 \to B_{31} \to B_{311}$.

Statistics (t/n) after iteration 1: R: 45/10, A: 7/2, A_1: 3/1, A_2: 0/0, A_3: 0/0, B: 34/7, B_1: 2/1, B_2: 4/1, B_3: 19/4, B_{31}: 7/2, B_{311}: 0/1, B_{312}: 0/0, B_{32}: 5/1.

Iteration 2.

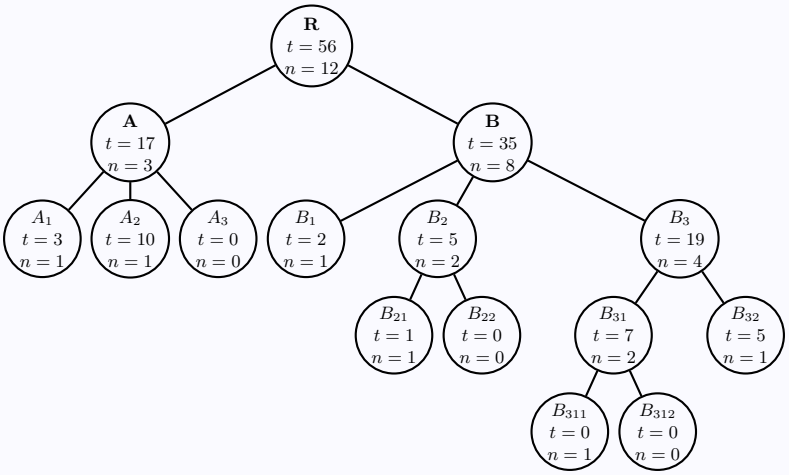
- *Selection* at R ($N = 10$): A: $\frac{7}{2} + 2\sqrt{\frac{\ln 10}{2}} = 5.646$; B: $\frac{34}{7} + 2\sqrt{\frac{\ln 10}{7}} = 6.004 \Rightarrow$ go to **B**.
- *Selection* at B ($N = 7$): B_1: $\frac{2}{1}+2\sqrt{\frac{\ln 7}{1}} = 4.790$; B_2: $\frac{4}{1}+2\sqrt{\frac{\ln 7}{1}} = 6.790$; B_3: $\frac{19}{4}+2\sqrt{\frac{\ln 7}{4}} = 6.145$ $\Rightarrow$ go to B_2.
- *Expansion*: B_2 was visited before ($n > 0$) and has no children $\Rightarrow$ add B_{21}, B_{22}; roll out from the first one, B_{21}.
- *Simulation*: rollout from B_{21} returns 1.
- *Backpropagation*: add 1 to t and 1 to n for R $\rightarrow$ B $\rightarrow B_2 \rightarrow B_{21}$.

Statistics (t/n) after iteration 2: R: 46/11, A: 7/2, A_1: 3/1, A_2: 0/0, A_3: 0/0, B: 35/8, B_1: 2/1, B_2: 5/2, B_{21}: 1/1, B_{22}: 0/0, B_3: 19/4, B_{31}: 7/2, B_{311}: 0/1, B_{312}: 0/0, B_{32}: 5/1.

Iteration 3.

- *Selection* at R ($N = 11$): A: $\frac{7}{2} + 2\sqrt{\frac{\ln 11}{2}} = 5.690$; B: $\frac{35}{8} + 2\sqrt{\frac{\ln 11}{8}} = 5.470 \Rightarrow$ go to **A**.
- *Selection* at A ($N = 2$): A_1: $\frac{3}{1} + 2\sqrt{\frac{\ln 2}{1}} = 4.665$; A_2: ∞ (unvisited); A_3: ∞ (unvisited) $\Rightarrow$ go to A_2.
- *Expansion*: none — A_2 has $n = 0$, so we roll out directly from it.
- *Simulation*: rollout from A_2 returns 10.
- *Backpropagation*: add 10 to t and 1 to n for R $\rightarrow$ A $\rightarrow A_2$.

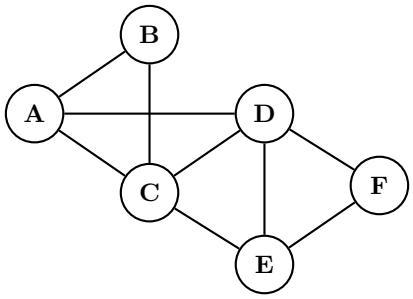
Statistics (t/n) after iteration 3: R: 56/12, A: 17/3, A_1: 3/1, A_2: 10/1, A_3: 0/0, B: 35/8, B_1: 2/1, B_2: 5/2, B_{21}: 1/1, B_{22}: 0/0, B_3: 19/4, B_{31}: 7/2, B_{311}: 0/1, B_{312}: 0/0, B_{32}: 5/1.
Final tree:



(b) Which root action is chosen by the *max child*, *robust child* and *secure child* rules (secure: maximise $\bar{V}_i - C\sqrt{\ln N/n_i}$, $C = 2$)? What does *max-robust child* recommend?

Averages A: 5.67, B: 4.38 $\Rightarrow$ max child **A**; visits A: 3, B: 8 $\Rightarrow$ robust child **B**; lower bounds A: 3.85, B: 3.26 $\Rightarrow$ secure child **A**. Max and robust disagree, so max-robust says to keep searching until one child is best in both.

Question 8. This problem is about map colouring. Each region must be coloured Red (R), Green (G) or Blue (B), and neighbouring regions must have different colours. The constraint graph is shown below. The following questions are mutually independent.



(a) Cross out all values that would be eliminated by *forward checking* after variable B has been assigned as shown.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	R G B	**R**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	~~R~~ G B	**R** ~~G B~~	~~R~~ G B	R G B	R G B	R G B

Only the neighbours of B (A, C) lose R.

(b) A and B have been assigned as shown, but no constraint propagation has been done yet. **Cross out** all values that would be eliminated by *arc consistency* (AC-3).

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	**G**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R** ~~G B~~	~~R~~ **G** ~~B~~	~~R G~~ B	~~R~~ G ~~B~~	R ~~G B~~	~~R G~~ B

Propagation continues through the graph (unlike forward checking): every arc $X_i \to X_j$ is revised until no domain changes. Here arc consistency even leaves a single value for every variable.

(c) Variable A is already assigned and constraint propagation has been done. **Circle** all unassigned variables that might be selected by the *Minimum-Remaining-Values* (MRV) heuristic.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	G B	G B	R G B	R G B

Remaining values: B: 2, C: 2, D: 2, E: 3, F: 3 ⇒ MRV may select **B, C, D**.

(d) Same situation. **Circle** all unassigned variables that might be selected by the *Degree Heuristic*.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	G B	G B	R G B	R G B

Degree = number of constraints with other *unassigned* variables: B: 1, C: 3, D: 3, E: 3, F: 2 ⇒ the degree heuristic may select **C, D, E**.

(e) Assume no variable has been assigned yet. Solve the CSP using backtracking with *forward checking*, choosing variables with MRV, then the degree heuristic (DH), and finally alphabetical order; try values in the order R, G, B. For every step give the MRV value of every region not yet coloured, and the DH values of the regions tied on the minimum MRV.

Step 1

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	3	3	3	3	3	3
DH	3	2	4	4	3	2

Assign to the variable **C** with the value **R** (tie on MRV and DH, broken alphabetically). Domains before this step: A=RGB, B=RGB, C=RGB, D=RGB, E=RGB, F=RGB.
Step 2

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	2	–	2	2	3
DH	2	1		3	2	

Assign to the variable **D** with the value **G** (tie on MRV broken by DH). Domains before this step: A=GB, B=GB, C=R, D=GB, E=GB, F=RGB.
Step 3

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	1	2	–	–	1	2
DH	1				1	

Assign to the variable **A** with the value **B** (tie on MRV and DH, broken alphabetically). Domains before this step: A=B, B=GB, C=R, D=G, E=B, F=RB.
Step 4

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	–	1	–	–	1	2
DH		0			1	

Assign to the variable **E** with the value **B** (tie on MRV broken by DH). Domains before this step: A=B, B=G, C=R, D=G, E=B, F=RB.
Step 5

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	–	1	–	–	–	1
DH		0				0

Assign to the variable **B** with the value **G** (tie on MRV and DH, broken alphabetically). Domains before this step: A=B, B=G, C=R, D=G, E=B, F=R.
Step 6

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	–	–	–	–	–	1
DH						

Assign to the variable **F** with the value **R** (unique minimum MRV). Domains before this step: A=B, B=G, C=R, D=G, E=B, F=R.
Solution: A=B, B=G, C=R, D=G, E=B, F=R. No backtracking is needed.

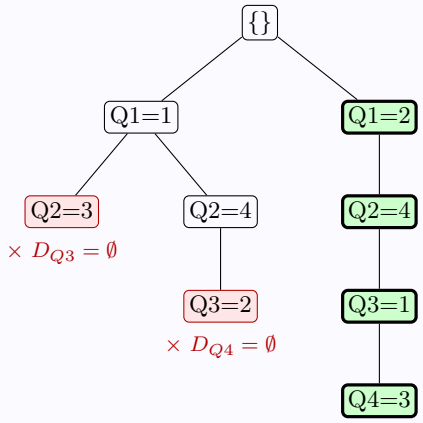
Question 9. 4-queens as a CSP: variable Q_i is the row of the queen in column i, with domain $\{1, 2, 3, 4\}$.

(a) Write the constraints.

For all $i < j$: $Q_i \neq Q_j$ and $|Q_i - Q_j| \neq j - i$ (columns are distinct by construction).

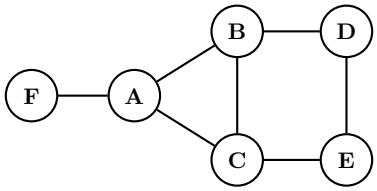
(b) Solve with backtracking and *forward checking*, variables in the order $Q_1, \ldots, Q_4$, values in increasing order. Give the domains after every assignment and draw the search tree, marking the wipe-outs.

Step	Assign	Q1	Q2	Q3	Q4	Note
1	Q1=1	**1**	34	24	23	
2	Q2=3	*1*	**3**	$\emptyset$	23	D_{Q3} wiped out
backtrack: undo Q2=3						
3	Q2=4	*1*	**4**	2	3	
4	Q3=2	*1*	*4*	**2**	$\emptyset$	D_{Q4} wiped out
backtrack: undo Q3=2						
backtrack: undo Q2=4						
backtrack: undo Q1=1						
5	Q1=2	**2**	4	13	134	
6	Q2=4	*2*	**4**	1	13	
7	Q3=1	*2*	*4*	**1**	3	
8	Q4=3	*2*	*4*	*1*	**3**	



Solution: $Q_1 = 2$, $Q_2 = 4$, $Q_3 = 1$, $Q_4 = 3$ (its mirror image $(3, 1, 4, 2)$ is the other solution).

Question 10. The constraint graph below has only binary constraints and no variable is assigned initially. For each independent scenario, list every variable whose domain *might* change.



(a) A value is assigned to A and forward checking is run for A.

Neighbours of A: **B, C, F**.

(b) A is assigned and forward checking is run; then B is assigned and forward checking is run for B.

Unassigned neighbours of B: **C, D**.

(c) A is assigned and arc consistency is enforced.

Propagation reaches the whole connected component: **B, C, D, E, F**.

(d) A is assigned and arc consistency is enforced; then B is assigned and arc consistency is enforced again.

C, D, E: propagation from B continues through unassigned variables only. F is reachable only through A, whose single value is already consistent, so A cannot change and no arc into F is re-examined.

Question 11. Answer the following questions briefly.

(a) Describe the "laws of thought" approach with a syllogism, and give the two obstacles the lecture mentions.

Irrefutable reasoning: *All men are mortal; Socrates is a man; so Socrates is mortal.* Obstacles: not all intelligent behaviour is mediated by logic, and solving a problem "in principle" differs from solving it in practice (intractability).

(b) On a 4-connected grid, compare the number of leaves of a depth-20 search tree with the number of distinct states within 20 steps (about $2d^2$). What does this say about tree-search vs graph-search?

$4^{20} \approx 10^{12}$ leaves vs ≈ 800 states: almost all nodes are redundant paths, so graph-search (remembering reached states) turns an intractable search into an easy one.

(c) In simulated annealing a successor is $\Delta E = -3$ worse than the current state. Compute its acceptance probability at $T = 10$, 3, 1. Why is simulated annealing better than plain hill climbing?

$T = 10$: $e^{-3/10} = 0.741$, $T = 3$: $e^{-3/3} = 0.368$, $T = 1$: $e^{-3/1} = 0.050$. Early random downhill moves let it escape local maxima; as T decreases it behaves like hill climbing.

(d) With perfect move ordering, how many nodes does alpha-beta examine? Evaluate the effective branching factor for chess ($b \approx 35$) and name two move-ordering techniques.

$O(b^{m/2})$, effective branching factor $\sqrt{35} \approx 5.9$. Killer-move heuristic; transposition tables.

Question 12. True or false? For each statement, circle **T** or **F** and justify your answer in one or two sentences (an answer without a correct justification earns no credit).

1. In graph search the frontier separates the expanded (interior) region from the unreached (exterior) region. **T** **F**

TRUE. Graph-search separation property: every path to an unreached state passes through the frontier.

2. For BFS, memory is a bigger problem than execution time. **T** **F**

TRUE. All generated nodes are kept: $O(b^d)$ memory runs out first.

3. A* graph search with a consistent heuristic never needs to re-expand a state. **T** **F**

TRUE. The first time a state is expanded it is on an optimal path, so no cheaper path is found later.

4. A* with an inadmissible heuristic can never return an optimal solution. **T** **F**

FALSE. It may, e.g. if all nodes on an optimal path have admissible h values.

5. For the 8-puzzle, $h = \max\{2, h_{\text{Manhattan}}\}$ is admissible. **T** **F**

FALSE. At the goal $h = 2 > 0$.

6. For the 8-puzzle, $h = \min\{2, h_{\text{Manhattan}}\}$ is admissible. **T** **F**

TRUE. $h \leq h_{\text{Manhattan}} \leq h^*$.

7. Pruning in expectimax is possible only if the leaf values are known to lie in a bounded range. **T** **F**

TRUE. Otherwise an unseen outcome could always raise the average above any bound.

8. In UCB1, increasing C makes MCTS more exploratory. **T** **F**

TRUE. C scales the exploration term $\sqrt{\ln N / n_i}$.

9. The least-constraining-value heuristic tries first the value that rules out the most choices for the neighbours. **T** **F**

FALSE. It prefers the value ruling out the *fewest* (fail-last).

10. Any constraint over more than two variables can be transformed into binary constraints. **T** **F**

TRUE. As stated in the lecture (e.g. by introducing auxiliary variables).